

Oscar Cortez

ITAI-3377 — Lab 03: Edge AI (MNIST TensorFlow Lite)

Houston Community College

Prof. Sitaram Ayyagari

February 5, 2026

L03 — Deploying a Simple AI Model on a Simulated Edge Device using Visual Studio Code

Objective

Deploy a simple AI model on a simulated edge device to understand the basics of edge computing and AI integration using Visual Studio Code (VS Code).

Environment / Tools

- Operating System: Windows 11
- IDE: Visual Studio Code (VS Code)
- Python: 3.12 (project folder: PY312)
- TensorFlow: Installed via `pip install tensorflow`
- Node.js: v24.13.0
- Edge Impulse CLI / Daemon: v1.37.2

Step 1: Set Up the Environment

1. Installed Python and Visual Studio Code.
2. Installed TensorFlow in VS Code terminal:
 - `pip install tensorflow`
3. Installed Node.js and Edge Impulse CLI:
 - `npm install -g edge-impulse-cli`
4. Verified versions (see screenshots/logs):
 - `node -v`
 - `edge-impulse-daemon --version`

Step 2: Prepare the Dataset (MNIST)

Loaded and preprocessed the MNIST dataset by normalizing pixel values to a 0–1 range and reshaping images to include a single channel (28×28×1). This formatting is required for the CNN input layer.

Oscar Cortez
ITAI-3377 — Lab 03: Edge AI (MNIST TensorFlow Lite)
Houston Community College
Prof. Sitaram Ayyagari
February 5, 2026

Step 3: Train a Simple AI Model

A simple convolutional neural network (CNN) model was created, compiled with the Adam optimizer, and trained on the MNIST dataset. Training was performed in VS Code using Python.

Step 4: Convert and Deploy the Model

The trained Keras model was converted to TensorFlow Lite format and saved as model.tflite using the TensorFlow Lite converter. This file represents an edge-deployable version of the model.

Edge Impulse Upload Attempt:

The assignment instructions include uploading the model to Edge Impulse and using the platform to simulate the edge device environment. An attempt was made to use Edge Impulse CLI on Windows; however, the Edge Impulse daemon repeatedly failed during authentication and terminated with a Windows-related uv async assertion error. Screenshots/logs are included to document this issue.

Step 5: Test and Validate the Model

To validate the TensorFlow Lite model locally (simulated edge-style runtime), model.tflite was loaded using a TFLite interpreter and tested on 25 MNIST samples. The model ran successfully and produced:

- **TFLite accuracy on 25 samples: 96.00%** (see screenshot/log)

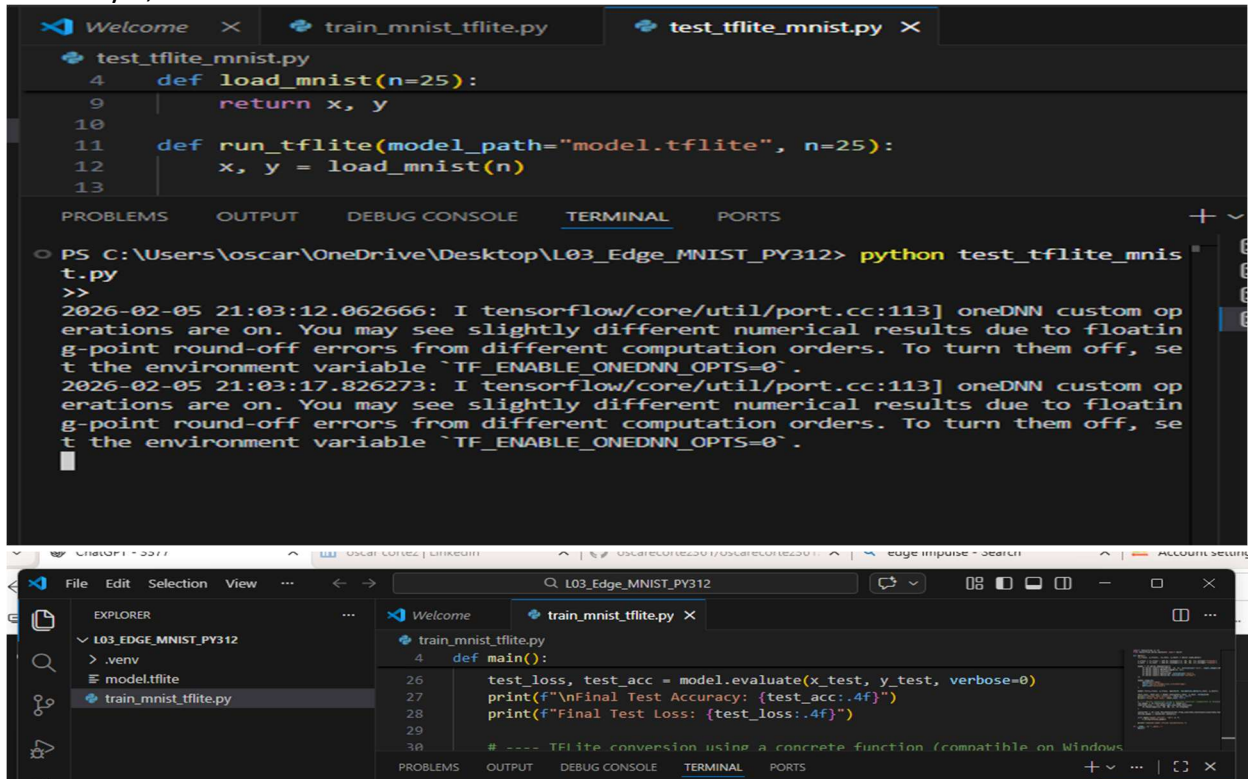
Results / Observations

- model.tflite was successfully generated.
- Local inference with TensorFlow Lite completed successfully.
- Accuracy result: **96.00% on 25 samples**.
- Edge Impulse CLI upload/simulation could not be completed on Windows due to daemon failure; evidence is included.

Conclusion

This lab successfully demonstrated the workflow of preparing MNIST data, training a simple CNN model in VS Code, converting it to TensorFlow Lite, and validating it via local inference. Although Edge Impulse simulation could not be completed due to a Windows daemon issue, the deployment-ready model (model.tflite) was created and tested with documented results.

Oscar Cortez
ITAI-3377 — Lab 03: Edge AI (MNIST TensorFlow Lite)
Houston Community College
Prof. Sitaram Ayyagari
February 5, 2026



The image shows a Visual Studio Code editor window with two tabs: `train_mnist_tflite.py` and `test_tflite_mnist.py`. The `test_tflite_mnist.py` tab is active, displaying the following Python code:

```
4 def load_mnist(n=25):
9     return x, y
10
11 def run_tflite(model_path="model.tflite", n=25):
12     x, y = load_mnist(n)
13
```

Below the code editor, the **TERMINAL** panel shows the output of running `python test_tflite_mnist.py` in a PowerShell prompt at `C:\Users\oscar\OneDrive\Desktop\L03_Edge_MNIST_PY312`. The output includes TensorFlow warnings about oneDNN custom operations and float point round-off errors, followed by a blank line.

```
PS C:\Users\oscar\OneDrive\Desktop\L03_Edge_MNIST_PY312> python test_tflite_mnist.py
>>
2026-02-05 21:03:12.062666: I tensorflow/core/util/port.cc:113] oneDNN custom operations are on. You may see slightly different numerical results due to floatin
g-point round-off errors from different computation orders. To turn them off, se
t the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-02-05 21:03:17.826273: I tensorflow/core/util/port.cc:113] oneDNN custom op
erations are on. You may see slightly different numerical results due to floatin
g-point round-off errors from different computation orders. To turn them off, se
t the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.

```

The bottom portion of the image shows the `train_mnist_tflite.py` file in the editor, with the following code visible:

```
4 def main():
26     test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
27     print(f"\nFinal Test Accuracy: {test_acc:.4f}")
28     print(f"Final Test Loss: {test_loss:.4f}")
29
30 # ---- TFlite conversion using a concrete function (compatible on Windows
```

The Explorer panel on the left shows the project structure for `L03_EDGE_MNIST_PY312`, including `.venv`, `model.tflite`, and `train_mnist_tflite.py`.

Oscar Cortez
ITAI-3377 — Lab 03: Edge AI (MNIST TensorFlow Lite)
Houston Community College
Prof. Sitaram Ayyagari
February 5, 2026

```
def load_mnist(n=25):
    (_, _), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
    x = x_test[:n].astype(np.float32) / 255.0
    y = y_test[:n].astype(np.int32) / 10
```

```
Sample 18: true=3 pred=5 conf=0.9063
Sample 19: true=4 pred=4 conf=1.0000
Sample 19: true=4 pred=4 conf=1.0000
Sample 20: true=9 pred=9 conf=0.9991
Sample 21: true=6 pred=6 conf=1.0000
Sample 21: true=6 pred=6 conf=1.0000
Sample 22: true=6 pred=6 conf=1.0000
Sample 22: true=6 pred=6 conf=1.0000
Sample 23: true=5 pred=5 conf=1.0000
Sample 24: true=4 pred=4 conf=1.0000

TFLite accuracy on 25 samples: 96.00%
PS C:\Users\oscar\OneDrive\Desktop\L03_Edge_MNIST_PY312>
```

```
correct += (pred == int(y[i]))
print(f"Sample {i:02d}: true={int(y[i])} pred={pred} conf={conf:.2f}")

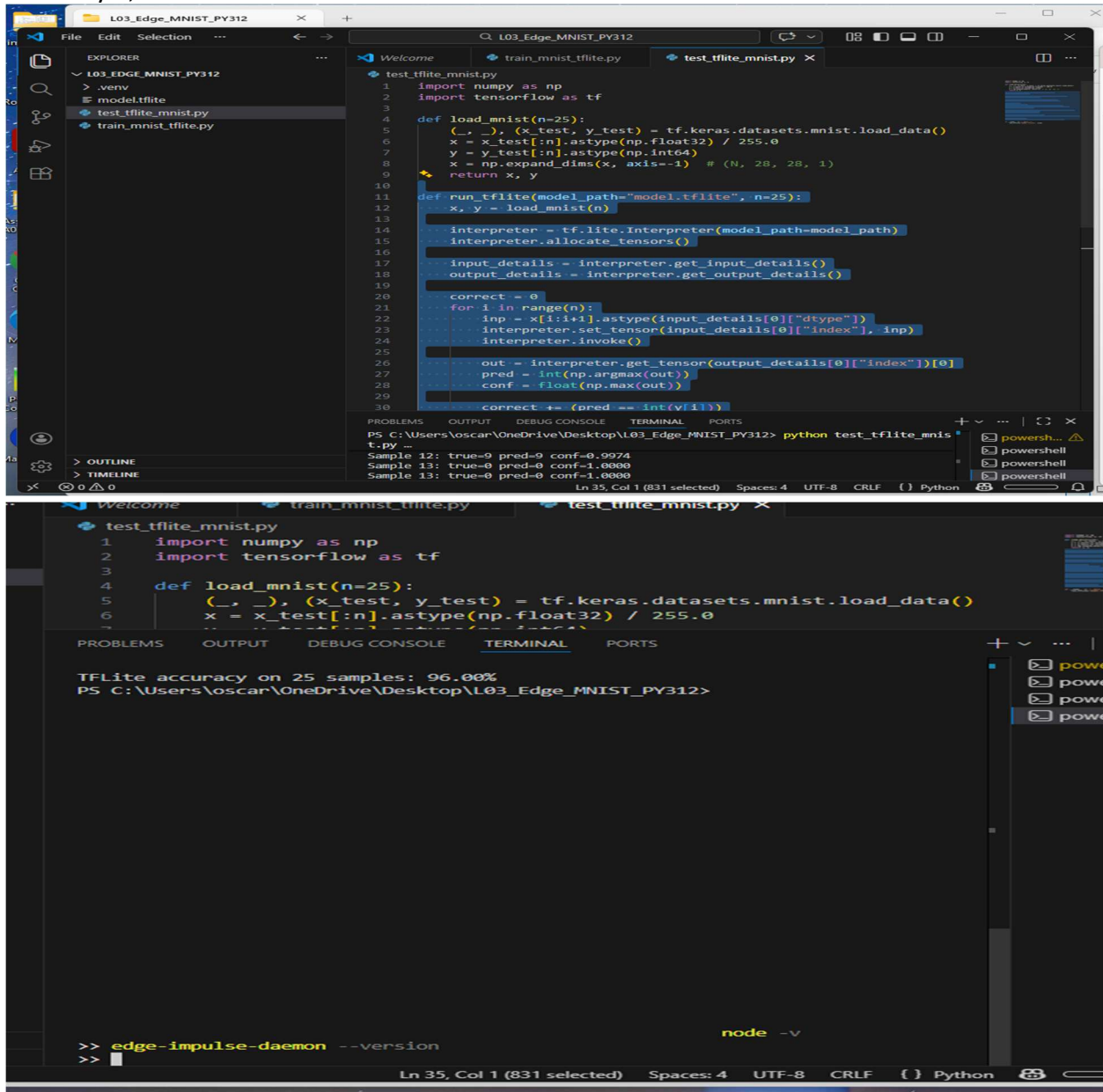
acc = correct / n
print(f"\nTFLite accuracy on {n} samples: {acc:.2f}")

if __name__ == "__main__":
    run_tflite("model.tflite", n=25)
```

```
PS C:\Users\oscar\OneDrive\Desktop\L03_Edge_MNIST_PY312> python test_tflite_mnist.py
>>
Sample 08: true=5 pred=5 conf=0.8613
Sample 09: true=9 pred=9 conf=1.0000
Sample 10: true=0 pred=0 conf=1.0000
Sample 11: true=6 pred=6 conf=1.0000
Sample 12: true=9 pred=9 conf=0.9974
Sample 13: true=0 pred=0 conf=1.0000
Sample 14: true=1 pred=1 conf=1.0000
Sample 15: true=5 pred=5 conf=1.0000
Sample 16: true=9 pred=9 conf=1.0000
Sample 17: true=7 pred=7 conf=1.0000
Sample 18: true=3 pred=5 conf=0.9063
Sample 19: true=4 pred=4 conf=1.0000
Sample 20: true=9 pred=9 conf=0.9991
Sample 21: true=6 pred=6 conf=1.0000
Sample 22: true=6 pred=6 conf=1.0000
Sample 23: true=5 pred=5 conf=1.0000
Sample 24: true=4 pred=4 conf=1.0000

TFLite accuracy on 25 samples: 96.00%
PS C:\Users\oscar\OneDrive\Desktop\L03_Edge_MNIST_PY312>
```

Oscar Cortez
ITAI-3377 — Lab 03: Edge AI (MNIST TensorFlow Lite)
Houston Community College
Prof. Sitaram Ayyagari
February 5, 2026



```
1 import numpy as np
2 import tensorflow as tf
3
4 def load_mnist(n=25):
5     (_, _), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
6     x = x_test[:n].astype(np.float32) / 255.0
7     y = y_test[:n].astype(np.int64)
8     x = np.expand_dims(x, axis=-1) # (N, 28, 28, 1)
9     return x, y
10
11 def run_tflite(model_path="model.tflite", n=25):
12     x, y = load_mnist(n)
13
14     interpreter = tf.lite.Interpreter(model_path=model_path)
15     interpreter.allocate_tensors()
16
17     input_details = interpreter.get_input_details()
18     output_details = interpreter.get_output_details()
19
20     correct = 0
21     for i in range(n):
22         inp = x[i:i+1].astype(input_details[0]["dtype"])
23         interpreter.set_tensor(input_details[0]["index"], inp)
24         interpreter.invoke()
25
26         out = interpreter.get_tensor(output_details[0]["index"])[0]
27         pred = int(np.argmax(out))
28         conf = float(np.max(out))
29
30         correct += (pred == int(y[i]))
31
32     return correct / n
```

PS C:\Users\oscar\OneDrive\Desktop\L03_Edge_MNIST_PY312> python test_tflite_mnist.py

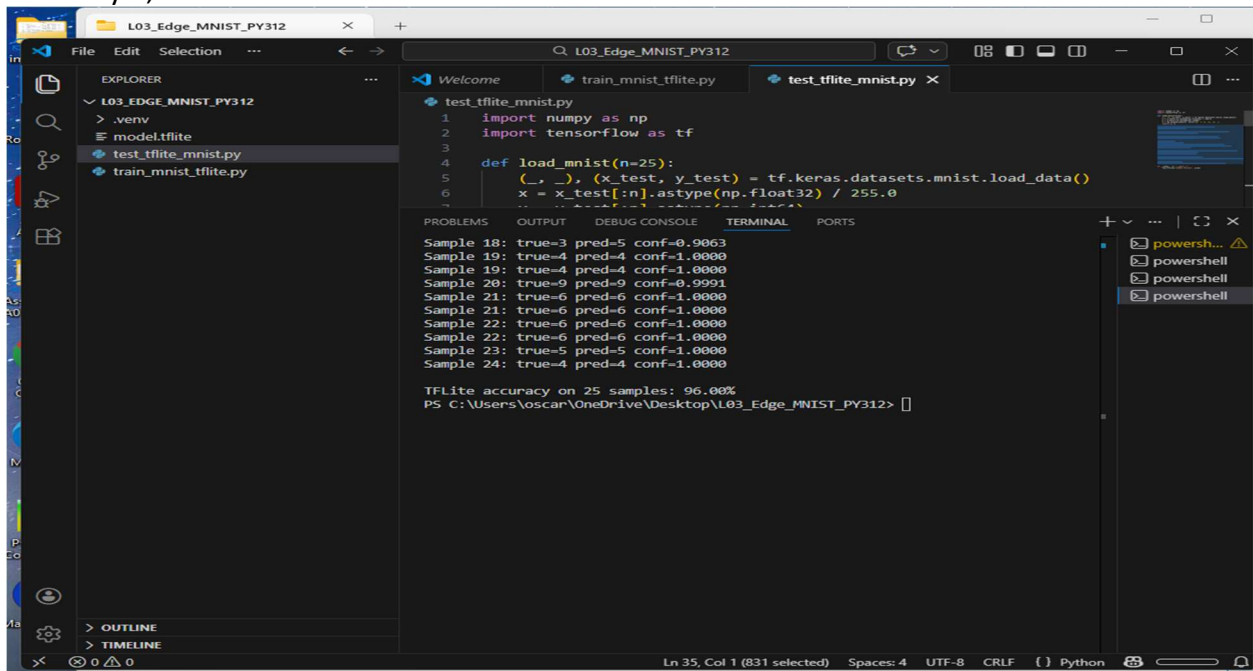
Sample 12: true=0 pred=0 conf=0.9974
Sample 13: true=0 pred=0 conf=1.0000
Sample 13: true=0 pred=0 conf=1.0000

TFLite accuracy on 25 samples: 96.00%

PS C:\Users\oscar\OneDrive\Desktop\L03_Edge_MNIST_PY312>

```
>>> edge-impulse-daemon --version
>>>
```

Oscar Cortez
ITAI-3377 — Lab 03: Edge AI (MNIST TensorFlow Lite)
Houston Community College
Prof. Sitaram Ayyagari
February 5, 2026



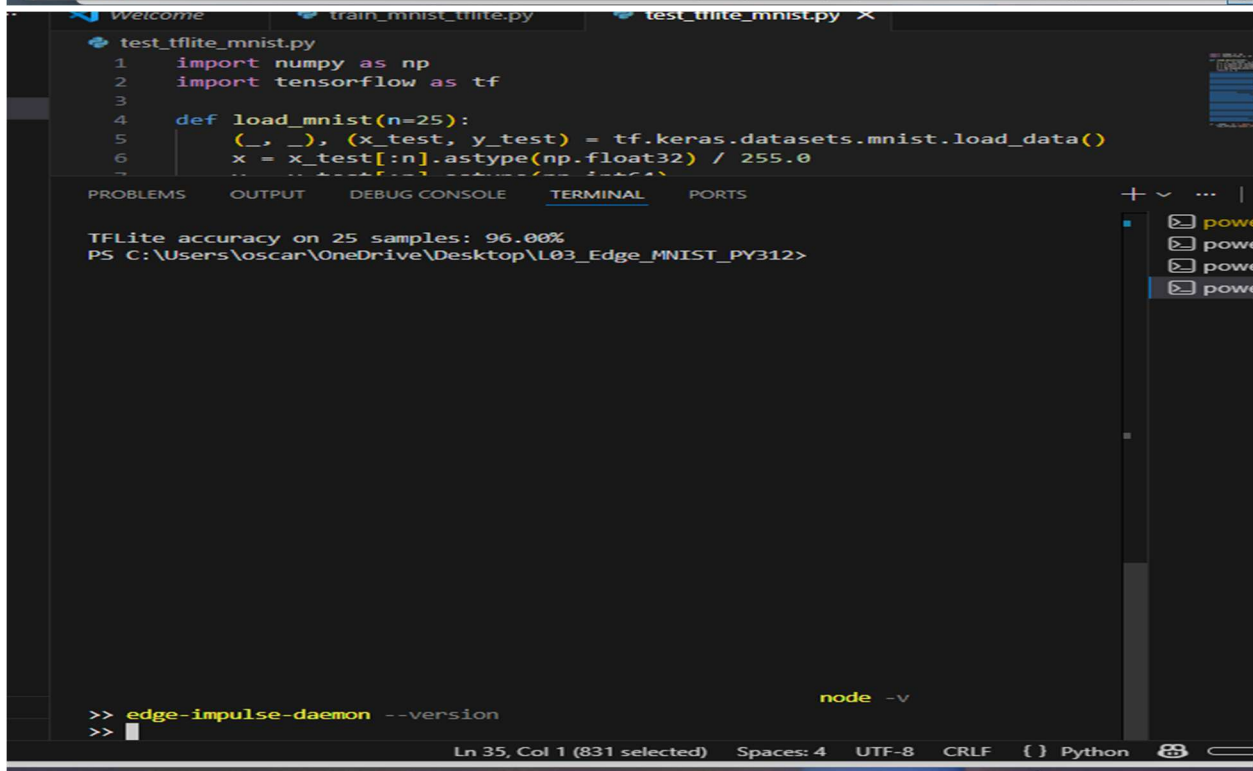
The screenshot shows the Visual Studio Code editor with the file `test_tflite_mnist.py` open. The file contains the following code:

```
1 import numpy as np
2 import tensorflow as tf
3
4 def load_mnist(n=25):
5     (_, _), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
6     x = x_test[:n].astype(np.float32) / 255.0
```

The terminal output shows the results of running the script:

```
Sample 18: true=3 pred=5 conf=0.9963
Sample 19: true=4 pred=4 conf=1.0000
Sample 19: true=4 pred=4 conf=1.0000
Sample 20: true=9 pred=9 conf=0.9991
Sample 21: true=6 pred=6 conf=1.0000
Sample 21: true=6 pred=6 conf=1.0000
Sample 22: true=6 pred=6 conf=1.0000
Sample 22: true=6 pred=6 conf=1.0000
Sample 23: true=5 pred=5 conf=1.0000
Sample 24: true=4 pred=4 conf=1.0000

TFLite accuracy on 25 samples: 96.00%
PS C:\Users\oscar\OneDrive\Desktop\L03_Edge_MNIST_PY312>
```



The screenshot shows the Visual Studio Code editor with the file `test_tflite_mnist.py` open. The file contains the following code:

```
1 import numpy as np
2 import tensorflow as tf
3
4 def load_mnist(n=25):
5     (_, _), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
6     x = x_test[:n].astype(np.float32) / 255.0
```

The terminal output shows the results of running the script:

```
TFLite accuracy on 25 samples: 96.00%
PS C:\Users\oscar\OneDrive\Desktop\L03_Edge_MNIST_PY312>
```

At the bottom of the terminal, there is a command prompt with the command `edge-impulse-daemon --version` and the output `node -v`.